

The AI Hub and the Collection

A short manual for the people who use the hub, and for the engineers who join the AI champions programme.

For	Everyone who signs in to the hub; AI champions; component owners; AI security engineers
Date	20 September 2026
Status	Reflects the repository at the date above; the readiness page in the repository is the live record

Before you start

You don't need to know how any of this works to use it. If you can sign in to a company website and use a search box, you can use the hub. This manual goes through the screens in the order you'll meet them, and then covers the two things that involve more than just using it: proposing an AI use for your team, and signing off a building block if you happen to own one or review them for security.

A word on what the hub is not. It is not a chatbot that can do things to your accounts or your systems. The assistant reads pages and tells you what they say, with the page cited. The agents can read tickets and deploy histories and draft things, but nothing they draft is posted or changed anywhere until a named person says yes. We built it that way on purpose, and the rest of this manual will show you where you can see that for yourself.

1. Signing in

Open the hub's address in your browser and sign in with the account you use for everything else. The hub never sees your password; the company's identity provider handles that and just tells the hub who you are and which groups you're in. What you see next depends on those groups. If something you expected is missing, that's not a fault. It's access, and there's a button on every card and page to ask for it.

2. The guide

Bottom right of every page there's a small button with a spark on it. That's the guide, and it's the fastest way to learn the hub. Open it once and it asks what you're here for: to build something, to understand and decide, or just to use the hub. Pick one; it suggests from your role, and you can change it any time from the top of the panel.

From then on the panel keeps to what matters for you. It tells you in a line what the page you're on is for. It suggests one next step, and only one, worked out from what you've actually done: a brief you left half-finished, components waiting on your signature, a page you haven't opened yet. It shows your path, a short list of steps that tick themselves off as you go, and a **Show me around** button that walks you through the pages with the relevant part of each one lit up. And you can ask it questions.

The screenshot shows the CrossRiver AI Hub interface. At the top, there's a navigation bar with 'Discover', 'My workspace', 'Build', and 'Learn'. A search bar is also present. The main content area is titled 'Incident first read agent'. It includes a description: 'An agent, complete: a template (role, stages, tools by tier, never list), tools reached only through the harness, a cited first read, one W1 proposal to confirm'. Below this, there are tabs for 'road R1', 'python', 'agent-ready', 'spec 55.1, 55.2, 55.5, 54.8, 54.5', 'agents', 'security', and 'governance'. There are buttons for 'Open in the portal' and 'Try in playground'. A 'What it does' section lists several points: 'The first thing a responder wants when an alert fires is a first read: what changed, what is failing, the leading hypothesis, with every claim pointing at a source, and at most one proposed action they can confirm or ignore. This is that agent, small enough to read in ten minutes, and the reference for what an agent in the collection is: a template that says what it is and may call, tools it reaches only through the harness, and a harness that enforces tiers, taint, budgets and the record. Copy it to start your own agent; replace the template and the fakes, keep the loop.', 'The catalog the harness loads is built and signed from the template's 'tools'; a call to anything else fails at the catalog hook.', 'A write (W1) parks with a hash of the exact tool and arguments; only the acting person confirms; the confirmation is consumed once.', 'A proposal is refused before any model call when the context is tainted, and the harness caps a tainted session at reads.', and 'The model sees projected, masked, fenced sources only; claims without a citation are dropped; a malformed answer is refused.' On the right, there's a 'Guide' sidebar with a 'WHERE YOU ARE' section, a 'SUGGESTED NEXT STEP' section, and a 'YOUR PATH' section with a progress indicator. The 'YOUR PATH' section shows a list of steps: 'What this is, in five minutes', 'What an agent cannot do', 'The controls, in plain terms', 'Who signs, and what they attest', 'What it costs and who uses it', 'The decisions that are yours', and 'Ask for a walkthrough'. The 'Get started' section at the bottom says 'Copy the directory. Edit 'TEMPLATE.md' first: the role sentence, the stages, the tools wit'.

The guide, open on an agent's listing for someone who chose 'understand and decide': where you are, one next step, the path, and a place to ask.

The answers are worth a word. The guide doesn't make things up about the platform. It reads our own documentation, quotes the passages that answer your question, and names the page each one came from, so you can go and read the rest. If the pages don't cover something, it says so rather than guessing. If a question looks like an instruction rather than a question, it declines. It has no tools and cannot act for you: it points, it explains, it never changes anything.

If you're a leader reading this, the guide is where to start. Choose **understand and decide**. On an agent's page it will tell you that the permission table you're looking at is the whole list and nothing outside it can run; on the sign-off page, who signs and what they attest; in your workspace, what it costs. Ask it plainly what an agent cannot do.

3. Discover

Discover is the catalog. The row at the top is what's already open to you. Below it, tabs sort everything else into assistants, agents, knowledge, tools, and the paved roads (the approved ways to build something new). The search box at the top finds any of them, and pressing Ctrl+K anywhere opens it.

CrossRiver AI Hub Discover My workspace Build Learn

Search assistants, tools, knowledge... [XK] [GK]

Gili Klainert · ops.lead · team-payments-ops 4 services available to you

What do you want to do with AI today?

Every assistant, agent and knowledge service here runs on the platform, so your identity, the bank's policy, a budget and an audit trail come with it. Nothing here is a side door.

Q Summarise this case, draft the R10 letter, find the late-file procedure... [XK]

Why did return R-1187 bounce? Draft an ACH return narration Late inbound file procedure Which tools can my role call?

Available to you by your role and channel · request what you cannot see

Employee assistant

Ask about policies, procedures and your own cases. Reads what your role can read; never writes.

R2 · read Internal GA

1,180 weekly users **Open**

Investigation triage

Triages payment returns and case notes; proposes fee reversals for your confirmation.

R2 · read ladder L1 GA

312 weekly users · 0.90 task success **Open**

Policy and procedures

Retrieval over runbooks, the policy manual and incidents, with provenance on every chunk.

R5 · retrieval Internal GA

2,400 queries a week **Open**

Compliance narration

Drafts return and recall narrations for a reviewer. Every draft is a labelled example.

R4 · drafts confidential preview

reviewer role needed Requested

Browse the catalog 9 listings · every one registered, evaluated and audited

All 9 Assistants **Agents** Knowledge Tools and APIs Roads and templates

Investigation triage

Triages payment returns and case

First responder

Page to postmortem in the on-call

Compliance narration

Drafts return and recall narrations for

Payments exception agent

Proposes hold releases and payee

Discover with the Agents tab open. Each card names the service, which road it runs on, whether it is generally available or still a preview, and how you reach it.

Each card says one of four things about access. **Open** means click it and go. **Request access** sends a request to your lead with one click. **View** means you can read about it but your role doesn't use it. **Contract** means another team runs it and the page says who to ask.

4. A listing page

Everything in the catalog has a page, and they all follow the same layout, so once you've read one you can read them all. Top to bottom: what it does; what it can read and do, each operation with its tier; the evidence behind it; the owner and how to get support; and the versions. The tier is worth a second look. **R** means read only. **W1** means it can write something, but only after the person using it confirms that exact action. **W2** means a second person has to approve. Anything involving money is simply not allowed.

Incident first read agent

An agent, complete: a template (role, stages, tools by tier, never list), tools reached only through the harness, a cited first read, one W1 proposal to confirm

road R1 python agent - ready spec \$5.1, \$5.2, \$5.5, \$4.8, \$4.5 agents security governance

Open in the portal → Try in playground

Template
TEMPLAT 2
role: the first read of an incident: an assistant that reads the t

Tools
cited-llm-engine, audit-chain

Harness
governed-action-loop
ladder L2

Language
python
standard library only

Status
ready
snapshot 2026-09-19

Pairs with
4
governed-action-loop, cited-llm-engine, teams-graph-connector, comm-templates

What it does

- ✓ The first thing a responder wants when an alert fires is a first read: what changed, what is failing, the leading hypothesis, with every claim pointing at a source, and at most one proposed action they can confirm or ignore. This is that agent, small enough to read in ten minutes, and the reference for what an agent in the collection is: a **template** that says what it is and may call, **tools** it reaches only through the harness, and a **harness** that enforces tiers, taint, budgets and the record. Copy it to start your own agent; replace the template and the fakes, keep the loop.
- ✓ The catalog the harness loads is built and signed from the template's `tools`; a call to anything else fails at the catalog hook.
- ✓ A write (W1) parks with a hash of the exact tool and arguments; only the acting person confirms; the confirmation is consumed once.
- ✓ A proposal is refused before any model call when the context is tainted, and the harness caps a tainted session at reads.
- ✓ The model sees projected, masked, fenced sources only; claims without a citation are dropped; a malformed answer is refused.

Sign-off v1.0.0 **owner pending; AI security pending**

Role	Status
Owner	pending pending
AI security	pending pending
Stage	built - next: use it once in a real project and record where (used_in, or the owner's sign-off form)

Owner sign-off on Incident first read agent 1.0.0

- ☐ I ran the tests and they are green python3 -m unittest discover -s tests -t . && python3 example.py
- ☐ I ran the live example python3 example.py
- ☐ I read the walkthrough end to end WALKTHROUGH.md

A listing page. On the right: the sign-off card, the getting-started steps, the owner and the versions.

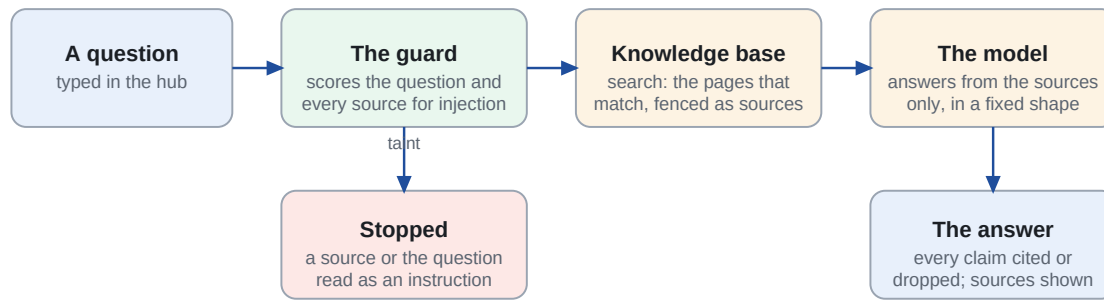
The chips under the title say at a glance which road it's on, what language it's written in, whether it's ready, and which parts of the platform design it implements. If both its owner and an AI security engineer have signed it, it says **signed**. If not, the hub calls it a preview, and you should treat it as one.

5. Asking the assistant

Open the assistant from Discover or from your workspace and type your question the way you'd ask a colleague. Three things are worth knowing about the answer you get back.

It cites its sources. Claims are marked in the text and the pages they came from are listed underneath, with their classification. A claim with no source is shown as unsupported, and you should read it as a hint rather than a fact. It shows its work. When it looks something up, that step appears in the conversation with what it searched for, so you can see what it did and not just what it said. And it asks one question at the end: did this answer yours? Please answer it. That one click is how we measure whether the thing is any good. If you'd rather talk to a person, the handoff button routes the conversation to one.

The assistant never changes a system. It reads pages and cites them; anything that looks like an instruction stops it before the model is called.



How an answer is produced. The knowledge base is searched, the pages that match are fenced off as sources, the model answers only from those, and every claim is either cited or dropped.

Now and then the assistant will tell you that a page looked like an instruction rather than evidence, and that it didn't use it. That's the protection doing its job: something in a page or a ticket was trying to steer it. Rephrase your question, or open the page directly.

6. Your workspace

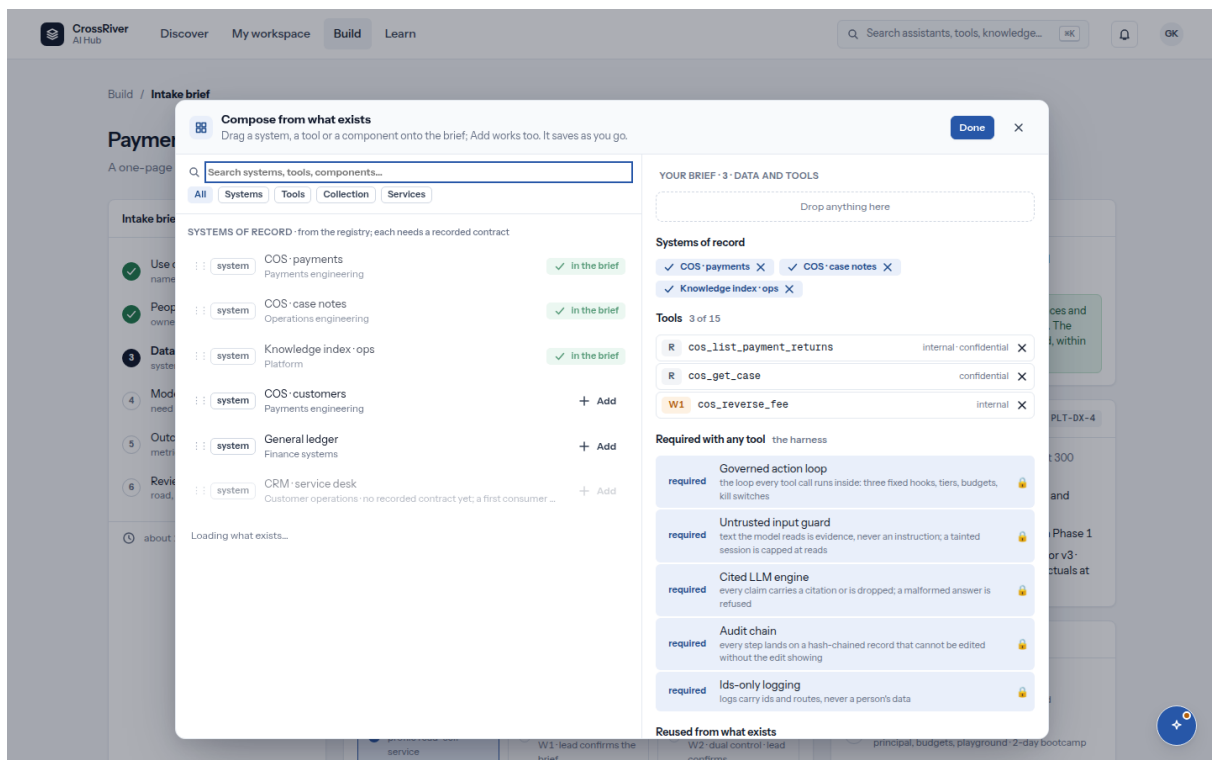
My workspace gathers what's yours: the assistants you use, your team's services and how far along each one is, your pending requests, your usage, and a sandbox playground for trying things. Settings are under the same menu (theme, density, language, accessibility, notifications). They apply straight away and follow your account.

7. Proposing an AI use for your team

The Build tab opens the intake brief. It's one page in six short sections and it saves as you go, so you can come back to it. It asks, in order: what the use case is and what happens today; who the business owner, product owner and domain expert are; which systems and tools it needs and what tier they'd run at; what kind of model it needs; one measurable outcome with today's figure and a target; and finally that you've read what happens next.

The third section is where you build from what already exists. Press **Browse the catalog** and a composer opens: on the left, everything the bank has (the systems of record, the tools each one offers with their tier, the collection's components, and the services already on the platform); on the right, your brief. Drag anything from the left onto the right, or press Add. A tool brings its system with it. Under the brief, the composer tells you what your choices mean as you make them: the highest tier so far and whether your ceiling covers it, the data classes the tools read, how many tools you have left under the session limit, and whether a lead will have to file the brief. One button fixes each of those. A system without a recorded contract is shown greyed with the reason, and a money tool is refused for a first consumer.

One thing you do not have to remember: the harness. The moment your brief names a tool, the composer shows five components locked under **Required with any tool**: the governed action loop, the untrusted input guard, the cited LLM engine, the audit chain and ids-only logging. That is the set every solution that calls a tool runs inside, and it is written into the brief when you file it, by the hub and again by the platform, so it cannot be left out or taken out. The review page names it as the brief's harness.



The composer, open from section 3. A tool has just been dropped onto the brief and brought its system with it; the panel below says what the choices mean and offers the fix.

As you fill it in, the brief estimates the monthly model spend and the review effort, and suggests which paved road fits. When you file it, the platform lead sees it. If what you're proposing would let an AI change a system rather than just read one, your team lead has to be the one to file it, and the form will tell you so.

8. If you're an AI champion

Champions are engineers, one per team, and you don't need to have built anything with AI before. The programme is a forty-five minute meeting every two weeks: ten minutes on what shipped, twenty on one deep dive, fifteen on the collection. You carry one initiative at a time. It starts as a brief, gets built from the collection on a paved road, and comes back as something the next team can reuse.

Your first week is small on purpose. Sign in to the hub. Run one live example from the Tools tab in a terminal and watch what it prints. Read one practice page in the knowledge base (the Learn tab links there). Then draft your brief and bring it to the next meeting. The technical guide is the next thing to read after that; it starts from the basics.

9. If you sign components off

A building block only reaches the shelf when two people have signed it at its current version. Its **owner**, who is named on it, signs by name. An **AI security engineer** signs by role. Both of you attest to four things on the form: you ran the tests and they were green, you ran the live example, you read the walkthrough all the way through, and you read the rules it enforces and its known limits. The owner also names the project it was first used in for real.

CrossRiver

AI Hub

Discover

My workspace

Build

Learn

Search assistants, tools, knowledge...

🔍

🔔

GK

Build / Sign-offs

Component sign-offs

A component reaches the shelf when its owner and an AI security engineer have each signed it at its current version. Sign here; the manifest in the repository is the record and the commit is the signature.

0 signed

23 awaiting owner

23 awaiting AI security

The queue

All 23OpenYours to sign 23Recorded here

COMPONENT	VERSION	STAGE	OWNER	AI SECURITY	
Audit chain tool · python · owner gil.klainert	1.0.0	built	pending	pending	Close

Owner sign-off on Audit chain 1.0.0

☐ I ran the tests and they are greenpython3 -m unittest discover -s tests -t . -v

☐ I ran the live examplepython3 example.py

☐ I read the walkthrough end to endWALKTHROUGH.md

☐ I read the rules it enforces and its known limitsREADME.md

Used once for real: which project

project name

The owner signs after one real use. Name the project; it is recorded in the manifest.

Note (optional)

Where it ran and what you changed to make it fit.

Sign as ownerrecorded as 1.0.0; the commit that follows is the signature

Comm templates
tool · python · owner gil.klainert

1.0.0

built

pending

pending

Sign

Governed action loop
tool · python · owner gil.klainert

1.0.0

built

pending

pending

Sign

Ids-only logging
tool · python · owner gil.klainert

1.0.0

built

pending

pending

Sign

You

You may sign as **owner** on the components that name you.

23 waiting on you. Before you sign: run the tests and the live example, read the walkthrough and the rules. The form asks you to attest to each.

Into the repository

0 recorded here

A sign-off recorded here is a request until it is in the manifest. Export the queue, apply it, and commit; the shelf tool re-runs the tests before it writes anything.

python3 tools/shelf.py --apply-signoffs shelf-signoffs.json
python3 tools/shelf.py --write
git commit -am "Sign off <component> <version>"

Download shelf-signoffs.json

The onboarding process

The sign-off queue under Build. The form opens in the row. The right column tells you what you may sign and what happens after you do.

The queue lists every component with its version and both sign-offs, and the **Yours to sign** filter shows what's waiting on you. A sign-off you record here is a request until an engineer applies it to the component and commits it; the download button on the page is that step, and the commit is what makes it a record. If a component gets a new version, both of you are asked again.

1 scaffolded
a manifest, placeholders

2 built
README, walkthrough,
example, tests green

3 used once
the owner names
the project

4 owner signed
by name, at this
version

5 security signed
by role, at this
version

6 on the shelf
GA on the hub and
in the knowledge base

A version bump returns a component to stage 3: both people sign the new version.

The form on the hub
four attestations, the first
real use; the server
refuses a missing one

The export
the queue's file, applied
by the shelf tool after
re-running the tests

The commit
the manifest is the record;
the commit is the
signature

A component's way to the shelf, and how a sign-off on the hub becomes a record in the repository.

Internal. Placeholders in capitals are not real identifiers. Nothing here is a credential.

CrossRiver AI Hub Discover My workspace Build Learn

Search assistants, tools, knowledge... WK Q GK

Build / Onboarding

Onboarding

Two things are onboarded here: components, on their way to the shelf, and people, into the roles that put them there. 0 of 23 on the shelf

A component's way to the shelf

1 scaffolded
a manifest from the scaffold: version 0.1.0, both sign-offs pending, a spec entry, a tag from the taxonomy

2 built
README, walkthrough, live example and tests filled and green; status set to ready

3 used once for real
used in one real project, named in the manifest (used_in) by the owner

4 owner signed
the owner signs at this version after running the tests and the example

5 AI security signed
an AI security engineer signs at this version after reading the rules and the walkthrough and running the example

6 on the shelf
listed as GA on Discover and in the knowledge base; a version bump, asks both to sign again

Built, not yet used for real

23

tests green; needs one real use, recorded by the owner

Audit chain 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
Comm templates 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
Governed action loop 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
Ids-only logging 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
RS256 jwt verify 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
Secrets by name 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
Stdlib http client 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
Typed API client 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
Untrusted input guard 1.0.0 · tool · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)
AWS sigv4 1.0.0 · integration · owner gil.klainert	next: use it once in a real project and record where (used_in, or the owner's sign-off form)

A new champion

week one

- 1 Read the programme page and the cadence
- 2 Sign in here, open Discover, and run one live example from the Tools tab
- 3 Start an initiative brief for your team's first use
- 4 Read the component onboarding page: stages, sign-offs, who signs
- 5 Bring one thing to the next meeting: a demonstration, a page, or a component scaffold

A component owner

by name

- 1 Your handle is the manifest's owner field; that is what lets you sign
- 2 Keep the README's known limits honest; bump the version on any change a consumer would notice

The onboarding tracker shows where every component is and what has to happen next, plus the checklists for a new champion, an owner and an AI security engineer.

10. Getting help

If	Then
You can't see a service you need	Use the request button on its card or page; your lead approves it
An answer looks wrong	Answer the feedback question with no, open the cited page, or use the handoff to reach a person
You don't know where to start	Open the guide (bottom right, or Alt+G) and tell it what you're here for
You want to propose something	Build, then start a brief
You're a champion and stuck	The champions channel, or the office hour named in the hub's footer
Something looks like a security problem	Tell the AI security engineers straight away. Don't wait for the next meeting